

Classes, Objects, Anonymous Objects, and Singleton Objects



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India





Learning Objectives

01

Understand OOP Concepts

Grasp the core ideas of Classes, Objects, Attributes, and Methods in Scala.

0

2

Create & Instantiate

Define classes, create objects, and instantiate them from blueprints.

03

Access Members

Work with data members and methods on class instances.

04

Advanced Object Types

Understand Anonymous Objects and Singleton Objects, and differentiate between Classes and Objects.





What is Object-Oriented Programming?

OOP organizes software around **Classes, Objects, Attributes, and Methods** — mirroring how we think about real-world entities.

Code Reusability

Write once, use many times across your application.

Modularity

Break complex systems into manageable, self-contained units.

Easy Maintenance

Isolate changes to specific classes without affecting the whole program.

Real-World Modeling

Map software structures directly to real-world concepts.



The Class (Blueprint)

Student

Attributes: Roll Number, Name, Department

Behaviors: Study(), AttendClass(), SubmitAssignment()

The Objects (Instances)

Each real student is a unique **object** created from the Student class blueprint.

- Student1 — Roll No: 101
- Student2 — Roll No: 102
- Student3 — Roll No: 103

All share the same structure but hold different data.

A **class** is a blueprint for creating objects. It defines variables (data members) and methods (behaviors).

Syntax

```
class ClassName {  
    // Variables  
    // Methods  
}
```

Example

```
class Student {  
    var name = "Rahul"  
  
    def display(): Unit = {  
        println(name)  
    }  
}
```

Student → Class | **name** → Data

Member | **display** → Method

Objects in Scala

An **object** is an instance of a class, created using the `new` keyword. Once created, you can access its members using dot notation.

```
class Student {  
  var name = "Anita"  
  def display(): Unit = { println("Name = " + name) }  
}
```

```
object Main {  
  def main(args: Array[String]): Unit = {  
    val s1 = new Student()  
    s1.display() // Output: Name = Anita  
  }  
}
```

Working with Multiple Objects

Each object is **independent** — they share the same class structure but maintain their own data. You can create as many instances as needed.

Code

```
class Student {  
    var name = ""  
    def display(): Unit = {  
        println(name)  
    }  
}
```

```
val s1 = new Student()  
val s2 = new Student()  
s1.name = "Alice"  
s2.name = "Bob"  
s1.display()  
s2.display()
```

Output

```
Alice  
Bob
```

Each object (s1, s2) holds its own name value independently, demonstrating object isolation.

What is an Anonymous Object?

An object created **without assigning it to a variable**. It is used immediately and discarded — ideal for one-time use scenarios.

Key Characteristics

- Used only once
- No variable name required
- Saves memory for temporary objects
- Syntax: `new`
`ClassName().method()`

```
class Student {  
    def display(): Unit = {  
        println("Anonymous Object")  
    }  
}
```

```
object Main {  
    def main(args: Array[String]):  
Unit = {  
        new Student().display()  
    }  
}
```

Singleton Objects

A **Singleton Object** is declared with the `object` keyword. Only **one instance** exists throughout the entire program — no instantiation needed.

Syntax & Example

```
object Calculator {  
  def add(a: Int, b: Int): Int = {  
    a + b  
  }  
}
```

```
println(Calculator.add(10, 20))  
// Output: 30
```

Why Use Singleton Objects?

- No need to create objects repeatedly
- Ideal for utility/helper methods
- Similar to **static methods** in Java
- Accessed directly by object name



Class vs. Singleton Object

Understanding the key differences helps you choose the right construct for each situation in your Scala programs.

Feature	Class	Singleton Object
Instance Creation	Required (new)	Not Required
Keyword	<code>class</code>	<code>object</code>
Multiple Instances	Yes	No — only one
Constructor	Available	Not Required





Program Demonstration: Student Management

This complete example combines a **class with a constructor**, a **Singleton Object** for utility, and an **Anonymous Object** — all working together.

```
class Student(var rollNo: Int, var name:
String) {
  def display(): Unit = {
    println("Roll No : " + rollNo)
    println("Name : " + name)
  }
}

object College {
  def welcome(): Unit = {
    println("Welcome to Scala Programming")
  }
}
```

```
object Main {
  def main(args: Array[String]): Unit = {
    College.welcome()
    val s1 = new Student(101, "Alice"); s1.display()
    val s2 = new Student(102, "Bob"); s2.display()
    new Student(103, "Charlie").display() // Anonymous
Object
  }
}
```



Expected Output

```
Welcome to Scala Programming
Roll No : 101
Name    : Alice
Roll No : 102
Name    : Bob
Roll No : 103
Name    : Charlie
```

Execution Steps

1 Singleton Call

`College.welcome()` — displays the welcome message.

2 Named Objects

Create `s1` (Alice, 101) and `s2` (Bob, 102) and call `display()`.

3 Anonymous Object

`new Student(103, "Charlie").display()` — used immediately, no reference stored.



OOP Foundations

Classes, Objects, Attributes, and Methods — the four pillars of OOP in Scala.



Anonymous Objects

One-time-use objects created and consumed immediately without a variable reference.



Classes & Objects

Defining blueprints with `class` and creating instances with `new`.



Singleton Objects

Program-wide single instances declared with `object`, similar to Java statics.

Key Concepts at a Glance

Class (blueprint)

Object (instance)

**Singleton
(single)**

These three constructs form the foundation of every Scala OOP program you will write.

Anonymous vs. Named vs. Singleton

01

Named Object:

- created with new
- stored in a variable
- reusable
- multiple instances

02

Anonymous Object:

- created with new
- no variable
- used once
- temporary

03

Singleton Object:

- declared with object keyword
- no instantiation
- one global instance
- utility methods

1

Activity 1 — Employee Class

Create a class `Employee` with attributes: Employee ID, Employee Name, Salary. Display employee information using a method.

2

Activity 2 — MathUtility Singleton

Create a Singleton Object `MathUtility` with methods `square()` and `cube()`. Call these methods directly.

3

Activity 3 — Anonymous Object

Create an Anonymous Object of any class and invoke a method directly without storing a reference.